

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : YUSUF HIBATULLAH

NIM : 109082500164

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Algoritma sorting digunakan untuk mengatur data ke dalam urutan tertentu, seperti dari nilai terkecil ke terbesar atau sebaliknya. Dengan data yang telah terurut, proses pencarian, analisis, dan pengolahan data dapat dilakukan dengan lebih cepat dan efisien. Terdapat berbagai jenis algoritma sorting yang memiliki metode dan tingkat efisiensi berbeda, di antaranya adalah Selection Sort dan Insertion Sort.

Selection Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara mencari elemen terkecil dari bagian data yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal bagian tersebut. Proses ini dilakukan secara berulang hingga seluruh data berada dalam urutan yang benar. Cara kerja Selection Sort cukup mudah dipahami karena pada setiap langkah algoritma akan memilih satu elemen terbaik untuk ditempatkan pada posisi yang sesuai. Meskipun sederhana, algoritma ini kurang efisien untuk data dalam jumlah besar karena memerlukan banyak proses perbandingan.

Sementara itu, Insertion Sort merupakan algoritma pengurutan yang bekerja dengan cara menyisipkan elemen ke posisi yang tepat pada bagian data yang telah terurut. Algoritma ini dimulai dari elemen kedua, kemudian membandingkannya dengan elemen sebelumnya hingga menemukan posisi yang sesuai. Proses tersebut dilakukan berulang sampai semua elemen tersusun secara terurut. Insertion Sort memiliki kelebihan dalam pengolahan data berukuran kecil dan data yang hampir terurut karena jumlah perpindahan data yang dilakukan relatif sedikit.

B. Guided

1. Selection Sort

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
```

```

arr[i]          arr[i], arr[idx_min] = arr[idx_min],
arr[i]
    }
}

func main() {
    var arrAngka [8]int
    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan angka pada indeks ke-%d
: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()
    fmt.Println("==== sebelum diurutkan ====")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("==== setelah diurutkan ====")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output :

```

PS D:\ALPRO LAPRAK\109082500164_YUSUF-HIB
Masukkan angka pada indeks ke-0 : 3
Masukkan angka pada indeks ke-1 : 7
Masukkan angka pada indeks ke-2 : 4
Masukkan angka pada indeks ke-3 : 1
Masukkan angka pada indeks ke-4 : 2
Masukkan angka pada indeks ke-5 : 3
Masukkan angka pada indeks ke-6 : 4
Masukkan angka pada indeks ke-7 : 5

==== sebelum diurutkan ====
3 7 4 1 2 3 4 5
==== setelah diurutkan ====
1 2 3 3 4 4 5 7
PS D:\ALPRO LAPRAK\109082500164_YUSUF-HIB

```

2. Insertion Sort

```
package main

import "fmt"

type mahasiswa struct {
    name string
    nim  string
    ipk  float64
}

type arrMhs [100]mahasiswa

func InsertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var j, i int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1

        for j >= 0 && T[j].name > temp.name {
            T[j+1] = T[j]
            j--
        }

        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("Masukkan data mahasiswa ke-", i+1)

        fmt.Print("Nama: ")
        fmt.Scan(&data[i].name)

        fmt.Print("NIM: ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&data[i].ipk)
    }
}
```

```

        fmt.Println("\nData mahasiswa sebelum diurutkan:")
        for i = 0; i < n; i++ {
            fmt.Println(data[i].name, data[i].nim,
data[i].ipk)
        }

        InsertionSortStruct(&data, n)

        fmt.Println("\nData mahasiswa setelah diurutkan:")
        for i = 0; i < n; i++ {
            fmt.Println(data[i].name, data[i].nim,
data[i].ipk)
        }
    }
}

```

Output :

```

Masukkan jumlah mahasiswa: 3
Masukkan data mahasiswa ke- 1
Nama: ABC
NIM: 001
IPK: 4
Masukkan data mahasiswa ke- 2
Nama: DEF
NIM: 002
IPK: 3.5
Masukkan data mahasiswa ke- 3
Nama: GHI
NIM: 004
IPK: 4

Data mahasiswa sebelum diurutkan:
ABC 001 4
DEF 002 3.5
GHI 004 4

Data mahasiswa setelah diurutkan:
ABC 001 4
DEF 002 3.5
GHI 004 4

```

C. Unguided

1. Soal Unguided 1 (INSERTION SORT)

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban :

```
package main

import "fmt"

func main() {
    var arr []int
    var x int

    // Input data sampai bilangan negatif
    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }

        arr = append(arr, x)
    }

    // Insertion Sort
    for i := 1; i < len(arr); i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }

    // Output array yang sudah diurutkan
    for i := 0; i < len(arr); i++ {
        fmt.Print(arr[i], " ")
    }

    fmt.Println()

    // Cek jarak antar data
    if len(arr) < 2 {
        fmt.Println("Data berjarak x")
        return
    }
}
```

```

    }

    jarak := arr[1] - arr[0]
    tetap := true

    for i := 2; i < len(arr); i++ {
        if arr[i]-arr[i-1] != jarak {
            tetap = false
            break
        }
    }

    if tetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :

```

31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\ALPRO LAPRAK\109082500164_YUSUF-HIBATULLAH>

```

Penjelasan :

Program ini menerima input bilangan bulat dari pengguna secara berulang hingga ditemukan bilangan negatif, yang berfungsi sebagai tanda berhenti. Semua bilangan non-negatif yang dimasukkan akan disimpan ke dalam slice arr.

Setelah input selesai, program mengurutkan elemen-elemen dalam slice menggunakan algoritma Insertion Sort, yaitu dengan mengambil setiap elemen dan menyisipkannya ke posisi yang tepat di antara elemen-elemen sebelumnya yang sudah terurut. Hasil pengurutan kemudian ditampilkan ke layar.

Terakhir, program menganalisis jarak antar elemen dalam array yang telah terurut. Jarak awal dihitung dari selisih elemen pertama dan kedua, lalu dibandingkan dengan selisih elemen-elemen berikutnya. Jika semua jarak antar elemen konsisten (sama), program mencetak "Data berjarak X" dengan nilai jaraknya. Jika tidak konsisten, program mencetak "Data berjarak tidak tetap".

2. Soal Unguided 2 (INSERTION SORT)

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan.

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakFavorit(pustaka DaftarBuku, n int) {
    max := pustaka[0]
```



```

        for i := 1; i < n; i++ {
            if pustaka[i].rating > max.rating {
                max = pustaka[i]
            }
        }

        fmt.Println("Buku Favorit:")
        fmt.Println(max.judul, max.penulis, max.penerbit,
max.tahun)
    }

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku
    var j int

    for i := 1; i < n; i++ {
        temp = pustaka[i]
        j = i - 1

        for j >= 0 && pustaka[j].rating <
temp.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }

        pustaka[j+1] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    fmt.Println("5 Buku Rating Tertinggi:")

    batas := 5
    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri := 0
    kanan := n - 1
    ketemu := false

    for kiri <= kanan && !ketemu {
        tengah := (kiri + kanan) / 2

        if pustaka[tengah].rating == r {

```

```

        fmt.Println("Data Buku Ditemukan:")
        fmt.Println(
            pustaka[tengah].judul,
            pustaka[tengah].penulis,
            pustaka[tengah].penerbit,
            pustaka[tengah].tahun,
            pustaka[tengah].eksemplar,
            pustaka[tengah].rating,
        )
        ketemu = true
    } else if r > pustaka[tengah].rating {
        kanan = tengah - 1
    } else {
        kiri = tengah + 1
    }
}

if !ketemu {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
}

func main() {
    var pustaka DaftarBuku
    var n, ratingCari int
    fmt.Scan(&n)

    DaftarkanBuku(&pustaka, n)
    fmt.Scan(&ratingCari)
    CetakFavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)
    CariBuku(pustaka, n, ratingCari)
}

```

Output :

```

3
B01 LaskarPelangi Andrea Hirata 10 2005 95
B02 Bumi TereLiye Gramedia 7 2014 88
B03 AtomicHabits JamesClear Penguin 5 2018 90
90
Buku Favorit:
LaskarPelangi Andrea Hirata 2005
5 Buku Rating Tertinggi:
LaskarPelangi
AtomicHabits
Bumi
Data Buku Ditemukan:
AtomicHabits JamesClear Penguin 2018 5 90

```

Penjelasan :

Program diawali dengan membaca jumlah data buku, kemudian setiap data buku disimpan ke dalam struct yang memiliki atribut seperti id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Setelah data dimasukkan, program akan menentukan buku favorit berdasarkan rating tertinggi. Selanjutnya data buku diurutkan secara menurun menggunakan metode insertion sort berdasarkan nilai rating. Setelah terurut, program menampilkan lima judul buku dengan rating tertinggi. Pada bagian akhir, program melakukan pencarian buku berdasarkan rating yang dimasukkan pengguna menggunakan metode binary search. Jika ditemukan, program akan menampilkan detail buku tersebut, sedangkan jika tidak ditemukan maka akan muncul pesan bahwa tidak ada buku dengan rating yang dicari.

3. Soal Unguided 3 (SELECTION SORT)

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer n ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }
    }
}
```

```

    }
    for j := 0; j < m-1; j++ {
        min := j

        for k := j + 1; k < m; k++ {
            if rumah[k] < rumah[min] {
                min = k
            }
        }

        rumah[j], rumah[min] = rumah[min],
rumah[j]
    }

    for j := 0; j < m; j++ {
        fmt.Print(rumah[j])

        if j != m-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()
}

```

Output :

```

3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program di atas menggunakan algoritma Selection Sort untuk mengurutkan nomor rumah kerabat Hercules secara membesar. Program pertama-tama membaca jumlah daerah, lalu membaca jumlah rumah dan nomor rumah di setiap daerah. Pada proses Selection Sort, program mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan posisi saat ini. Proses tersebut dilakukan berulang hingga seluruh data tersusun dari kecil ke besar. Setelah selesai diurutkan, hasil nomor rumah akan ditampilkan untuk masing-masing daerah.

4. Soal Unguided 4 (SELECTION SORT)

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        ganjil := []int{}
        genap := []int{}

        for j := 0; j < m; j++ {
            var x int
            fmt.Scan(&x)

            if x%2 == 1 {
                ganjil = append(ganjil, x)
            } else {
                genap = append(genap, x)
            }
        }

        for j := 0; j < len(ganjil)-1; j++ {
            min := j
```

```

        for k := j + 1; k < len(ganjil); k++ {
            if ganjil[k] < ganjil[min] {
                min = k
            }
        }

        ganjil[j], ganjil[min] = ganjil[min], ganjil[j]
    }

    for j := 0; j < len(genap)-1; j++ {
        max := j

        for k := j + 1; k < len(genap); k++ {
            if genap[k] > genap[max] {
                max = k
            }
        }

        genap[j], genap[max] = genap[max], genap[j]
    }

    for j := 0; j < len(ganjil); j++ {
        fmt.Print(ganjil[j], " ")
    }

    for j := 0; j < len(genap); j++ {
        fmt.Print(genap[j])

        if j != len(genap)-1 {
            fmt.Print(" ")
        }
    }

    fmt.Println()
}

```

Output :

```
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
```

Penjelasan :

Program di atas memisahkan nomor rumah ganjil dan genap ke dalam dua array berbeda. Bilangan ganjil diurutkan secara menaik menggunakan algoritma selection sort, sedangkan bilangan genap diurutkan secara menurun. Setelah proses pengurutan selesai, program menampilkan semua bilangan ganjil terlebih dahulu kemudian diikuti bilangan genap pada setiap daerah.

D. Kesimpulan

Pada praktikum ini telah dipelajari dua algoritma pengurutan data, yaitu Selection Sort dan Insertion Sort, menggunakan bahasa pemrograman Go. Selection Sort bekerja dengan cara mencari nilai terkecil pada bagian array yang belum terurut, kemudian menukarkannya ke posisi yang sesuai secara berulang hingga seluruh data terurut. Sementara itu, Insertion Sort bekerja dengan menyisipkan elemen satu per satu ke posisi yang benar pada bagian array yang sudah terurut sebelumnya.

Dari hasil praktikum dapat disimpulkan bahwa kedua algoritma mampu melakukan proses pengurutan data dengan baik, baik secara menaik maupun menurun. Selection Sort memiliki konsep yang sederhana dan mudah dipahami, namun membutuhkan proses pertukaran data pada setiap iterasi. Insertion Sort lebih efisien untuk data yang jumlahnya kecil atau data yang sebagian sudah terurut karena proses pergeseran datanya lebih sedikit.

